

BAIL RECKONER

Complete Project Documentation

Project Name	Bail Reckoner
Project Type	Rule-Based Legal Decision Support System
Architecture	REST API · MVC-style separation · Deterministic rule engine
Status	Feature-complete MVP (deployment-ready for academic/demo use)
Doc Version	3.0
Last Updated	April 2026

Abstract

Bail Reckoner is a deterministic legal-tech application for bail eligibility assessment and undertrial record management using statutory rules and risk factors. The system evaluates section severity, time served, and judicial risk factors, and returns explainable decisions with procedural requirements. It does not use AI/ML and focuses on transparent, auditable outcomes.

Table of Contents

- 1. Project Overview**
- 2. Technology Stack**
- 3. Features**
- 4. Project Structure**
- 5. Backend Documentation**
 - 5.1 server.js
 - 5.2 config/db.js
 - 5.3 Models
- 6. Controllers**
 - 6.1 bailController.js
 - 6.2 undertrialController.js
 - 6.3 legalSectionController.js
- 7. API Routes**
- 8. Data Layer & Seeding**
- 9. Frontend Documentation**
 - 9.1 API Layer — src/api/api.js
 - 9.2 Major Components
 - 9.3 App Shell (App.jsx)
- 10. UI/UX Characteristics**
- 11. Development Tools**
- 12. System Workflow**
- 13. Deterministic Evaluation Logic**
- 14. Validation Rules & Error Behavior**
- 15. Database Design**
- 16. System Characteristics**
- 17. Usage Examples**
- 18. Deployment**
- 19. Contributing**
- 20. Conclusion**

1. Project Overview

Bail Reckoner provides a structured, deterministic method for bail eligibility assessment. It maps submitted legal sections to statutory data, computes risk scores, applies the half-term rule, and returns a clear eligibility decision with procedural requirements.

Key Characteristics

- Fully rule-based and deterministic — same inputs always yield the same output
- No AI/ML or black-box inference — fully auditable logic
- MongoDB-backed legal sections (442 entries) and undertrial records
- Direct evaluation + undertrial record workflow support
- Frontend with responsive, mobile-aware UI and dark/light theme

2. Technology Stack

Layer	Technology / Version
Frontend	React 19.2.4 + Vite 8.0.0 + Framer Motion
Frontend Styling	CSS + Framer Motion animations
Backend Runtime	Node.js
Backend Framework	Express 5.2.1
Database	MongoDB Atlas
ODM	Mongoose 9.2.2
API Style	REST
Deployment	Vercel (separate client/server projects)

3. Features

Feature	Description
Direct Bail Evaluation	Instant eligibility assessment using statutory rules and risk factors
Undertrial Record Management	Create and store case records for later evaluation
Stored Record Evaluation	Load cases by ID and persist evaluation decisions
Legal Section Autocomplete	Live search through 442+ legal sections across 6 acts
Export Functionality	Download stored undertrial evaluations as JSON

Dark / Light Theme	Smooth theme switching with persistent preferences
Responsive Design	Mobile-friendly interface with smooth animations

4. Project Structure

```
Bail Reckoner/  
  client/          React frontend with Vite, components, API client, and styling  
  public/          logo.jpg, home.png, background.png, favicon.png  
  src/  
    api/api.js  
    components/  
      AboutPage.jsx  
      BailForm.jsx  
      UndertrialForm.jsx  
      UndertrialEvaluation.jsx  
    ui/            Button.jsx, Card.jsx, Carousel.jsx, FormField.jsx,  
                  Input.jsx, RiskSlider.jsx, SectionChips.jsx,  
StatusBadge.jsx  
  App.jsx, App.css, index.css  
  index.html, package.json  
  server/          Node.js/Express backend with MongoDB models, controllers, and  
data seeding  
  config/db.js  
  controllers/     bailController.js, legalSectionController.js,  
undertrialController.js  
  models/          LegalSection.js, Undertrial.js  
  routes/          bailRoutes.js, undertrialRoutes.js  
  data/legalSections.json  (442 offence rows, 6 acts)  
  seedLegalSections.js  
  server.js, vercel.json, package.json
```

5. Backend Documentation

5.1 server.js

Responsibilities
• Initialize Express application
• Configure CORS + JSON middleware
• Connect to MongoDB
• Register API routes
• Export app for Vercel serverless usage
• Start local server (PORT, default 5000)

5.2 config/db.js

Responsibilities
• Establish Mongoose connection
• Reuse existing connection when available
• Handle errors safely for local and serverless environments

5.3 Models

LegalSection.js

Attributes
• sectionNumber, act, offenceName, description, category
• maxPunishmentYears, bailable, compoundable
• requiresSurety, allowsPersonalBond, fineApplicable
• Unique index: (sectionNumber, act)

Undertrial.js

Attributes
• name, prisonerId, sections[], timeServedYears
• riskProfile.flightRisk, riskProfile.witnessRisk
• lastEvaluation.eligible, reason, evaluatedAt
• Unique index: (prisonerId)

6. Controllers

6.1 bailController.js

evaluateBail()

- Direct case evaluation from request payload

evaluateUndertrialBail()

- Evaluates stored undertrial by ID and persists lastEvaluation

Core Rule Logic

Steps
• Find matching legal sections from DB
• Determine highest punishment section
• Compute risk score (flightRisk + witnessRisk)
• Apply deterministic eligibility rules
• Return procedural requirements + section details

6.2 undertrialController.js

Functions
• createUndertrial() — validates and stores undertrial record
• getUndertrialById() — fetches undertrial record by ID

6.3 legalSectionController.js

Functions
• listLegalSections() — section search for input suggestion/autocomplete

7. API Routes

Bail Evaluation

Method	Endpoint	Purpose
POST	/api/bail/evaluate	Direct bail eligibility evaluation
POST	/api/bail/evaluate-undertrial/:id	Evaluate stored undertrial record

GET	/api/bail/legal-sections	Search legal sections with autocomplete (query param: q)
------------	--------------------------	--

Undertrial Management

Method	Endpoint	Purpose
POST	/api/undertrial/create	Create new undertrial record
GET	/api/undertrial/:id	Fetch undertrial record by ID

8. Data Layer & Seeding

server/data/legalSections.json is the source for the LegalSection MongoDB collection. It is a JSON array of 442 offence rows with comprehensive legal metadata. An older snapshot lives at server/data/legalSectionsOld.json (not used by the seed script).

Act	Sections	Notes
IPC	379	Indian Penal Code — largest share of entries
IT Act	21	Information technology / cyber-related offences
POCSO Act	19	Protection of Children from Sexual Offences Act
NDPS Act	15	Narcotic Drugs and Psychotropic Substances Act
SC/ST Act 1989	5	Scheduled Castes and Scheduled Tribes (Prevention of Atrocities) Act
Foreigners Act 1946	3	Foreign nationals — entry, stay, and document-related offences

Each Legal Section Includes

- Section number and act reference
- Offence name and detailed description
- Bail status (bailable / non-bailable)
- Punishment details (max years, life/death penalties)
- Procedural requirements (surety, personal bond, fines)
- Category classification (cyber crime, crimes against women, etc.)

Seeding

seedLegalSections.js imports the section dataset into MongoDB. Run using:

```
cd server
npm run seed
```

9. Frontend Documentation

9.1 API Layer — src/api/api.js

Functions
• evaluateBail(data) — POST /api/bail/evaluate
• createUndertrial(data) — POST /api/undertrial/create
• evaluateUndertrial(id) — POST /api/bail/evaluate-undertrial/:id
• listLegalSections(q) — GET /api/bail/legal-sections

9.2 Major Components

Component	Purpose
BailForm.jsx	Direct evaluation input form + decision result area
UndertrialForm.jsx	Create undertrial record + copy generated ID
UndertrialEvaluation.jsx	Evaluate by undertrial ID, display result
AboutPage.jsx	Dedicated About page with project context
Carousel.jsx	Single-card navigation between the three main workflows
components/ui/*	Reusable UI primitives: Button, Card, FormField, Input, RiskSlider, SectionChips, StatusBadge

9.3 App Shell (App.jsx)

Features
• Hero section with scroll-linked animation
• Drawer menu navigation
• Help button routing to About page
• Home button routing back to main evaluation interface

10. UI/UX Characteristics

Design Principles
• Card-based workflow layout
• Responsive for desktop and mobile
• Mobile-specific readability and performance tuning
• Loading, empty, and error states handled throughout
• Styled legal decision blocks and procedural requirement pills
• Framer Motion animations for transitions and hero section
• Dark/Light theme with persistent user preference

11. Development Tools

Area	Tool / Detail
Frontend Dev Server	Vite with HMR (Hot Module Replacement)
Frontend Linting	ESLint with React Hooks & Refresh plugins
Backend Dev Server	Nodemon for auto-restart on file changes
DB Connection	Mongoose connection management with reuse
Code Quality	ESLint configuration for consistent code standards

12. System Workflow

Direct Bail Evaluation

- Step 1.** User enters sections, time served, and risk values via BailForm
- Step 2.** Frontend calls POST /api/bail/evaluate
- Step 3.** Backend resolves matching legal sections from MongoDB
- Step 4.** Rule engine computes eligibility and procedural requirements
- Step 5.** Frontend renders decision, highest section details, and description

Undertrial Workflow

- Step 1.** User creates undertrial record via POST /api/undertrial/create
- Step 2.** MongoDB stores record with unique prisonerId
- Step 3.** User submits undertrial ID via UndertrialEvaluation component
- Step 4.** Backend fetches record, runs evaluation, updates lastEvaluation
- Step 5.** Updated result is returned and rendered in the frontend

13. Deterministic Evaluation Logic

#	Step	Detail
1	Validate payload	sections (non-empty array), timeServedYears (non-negative), flightRisk & witnessRisk (0–10)
2	Fetch legal sections	Match submitted section numbers against MongoDB legalSections collection
3	Determine section stats	Identify highest punishment section, maxPunishmentYears, all-bailable status
4	Compute risk score	$\text{riskScore} = \text{flightRisk} + \text{witnessRisk}$
5	Decision (ordered)	riskScore > threshold → Not eligible All bailable → Eligible Half-term served → Eligible Otherwise → Not eligible
6	Procedural requirements	Aggregate surety, personal bond, and fine applicability from matched sections

14. Validation Rules & Error Behavior

14.1 Validation Rules

Input Constraints
• sections: non-empty array of section identifiers
• timeServedYears: number, must be non-negative
• flightRisk: number in range 0–10
• witnessRisk: number in range 0–10

14.2 Error Behavior

Status Code	Meaning
400	Invalid input or validation failure
404	No matching legal section or missing undertrial record
500	Server-side failure

15. Database Design

Collection	Purpose
legalsections	Statutory section master data (442 rows seeded from legalSections.json)
undertrials	Undertrial records including lastEvaluation results

16. System Characteristics

Properties
• Rule-based — all decisions follow explicit statutory rules
• Deterministic — same inputs always yield the same output
• Explainable and auditable — no black-box inference
• Extensible legal section dataset via JSON (currently 442 entries across 6 acts)
• Modular backend controllers and reusable frontend components

17. Usage Examples

Direct Evaluation

Enter legal sections (e.g., IPC 420, IPC 379), time served, criminal records, and risk scores to get instant bail eligibility assessment with procedural requirements.

Record Management

Create undertrial records with prisoner details and legal sections, then evaluate them later using unique MongoDB ObjectIds. Evaluation results are persisted for future reference.

Legal Research

Use the autocomplete feature to quickly find relevant legal sections across IPC, IT Act, POCSO, NDPS, SC/ST Act, and Foreigners Act. Each section includes full metadata.

Export

Download stored undertrial evaluation results as JSON files for record keeping or integration with external systems.

18. Deployment

Component	Detail
Backend	Vercel serverless deployment (server/ directory as root)
Frontend	Vercel static deployment (client/ directory as root)

Environment Variables
• Backend: MONGO_URI — MongoDB Atlas connection string
• Frontend: VITE_API_BASE_URL — deployed backend URL (no trailing slash)
• Frontend: VITE_REPO_URL — GitHub repository URL (optional)

19. Contributing

Bail Reckoner is an educational legal-tech project. The application uses deterministic, rule-based algorithms to ensure transparency and auditability of bail evaluation logic.

For feedback or suggestions, use the "Send feedback" option in the app menu.

20. Conclusion

Bail Reckoner delivers consistent, explainable bail eligibility outcomes through deterministic statutory logic. By combining legal section mapping, risk score computation, half-term evaluation, and procedural output aggregation into a structured workflow, it serves as a reliable decision-support tool.

The system is well-suited for legal-tech demonstrations, academic projects, and decision-support contexts where transparency, auditability, and correctness are paramount.

End of Documentation — v3.0 · April 2026